

---

# **SWHV CCN4 Zenithal WCS and HPC Validation Note**

SWHV-ROB-TN-001-CCN4 v1.0

SWHV Team



SWHV-ROB-TN-001-CCN4 - Version 1.0 - 2026-03-22

# Contents

|       |                                                                   |    |
|-------|-------------------------------------------------------------------|----|
| 1     | Summary                                                           | 2  |
| 2     | The validator                                                     | 4  |
| 2.1   | JHV behavior modeled by the validator . . . . .                   | 4  |
| 2.2   | Astropy-based validation scope . . . . .                          | 6  |
| 2.3   | How to use the validator . . . . .                                | 6  |
| 2.3.1 | Astropy-based validation modes . . . . .                          | 7  |
| 2.3.2 | Internal JHV comparison modes . . . . .                           | 9  |
| 3     | Results                                                           | 11 |
| 3.1   | FITS projections . . . . .                                        | 11 |
| 3.2   | Astropy validation of JHV HPC rendering . . . . .                 | 12 |
| 3.3   | Formal-TAN versus JHV HPC . . . . .                               | 14 |
| 3.4   | Simple-TAN versus JHV HPC . . . . .                               | 14 |
| 4     | Appendix A: Theoretical Orthographic vs HPC discrepancy           | 16 |
| 5     | Appendix B: Idealized TAN small-angle discrepancy relative to HPC | 19 |

## List of Figures

|     |                                                                                        |    |
|-----|----------------------------------------------------------------------------------------|----|
| 4.1 | Orthographic vs HPC discrepancy across the projected solar disk at 1 AU . . . . .      | 17 |
| 4.2 | Maximum Orthographic vs HPC discrepancy versus observer distance . . . . .             | 18 |
| 5.1 | Maximum idealized TAN small-angle discrepancy relative to HPC versus observer distance | 20 |

id: 9781b058565e974bfb56ea214a97d250adfebe80

# 1 Summary

This note describes the zenithal WCS and `HPC` work implemented in JHV, and the corresponding validation against Astropy and direct internal comparison tests.

In this work, a new `TAN` path was implemented together with `AZP` and six-term `ZPN`, in line with the theoretical foundation of Greisen and Calabretta, “Representations of celestial coordinates in FITS” ([A&A 395, 1077-1122, 2002](#)). Hereafter, `formal-TAN` denotes this Greisen & Calabretta `TAN` formulation in JHV.

The JHV `TAN` implementation of previous versions is hereafter denoted as `simple-TAN`. It treats the orthographic surface point’s planar  $(x, y)$  coordinates as a small-angle approximation to helioprojective coordinates. It then feeds those coordinates directly into the linear image transform, instead of first converting the 3D point to helioprojective angles and then applying the `formal-TAN` world-to-plane projection.

An `HPC` projection mode was also implemented in JHV and validated as part of the same work; its solar-coordinate interpretation follows Thompson, “Coordinate systems for solar image data” ([A&A 449, 791-803, 2006](#)). Because `HPC` is fundamentally tied to a specific observer viewpoint, it is not a natural fit for the multi-viewpoint aspect of JHV.

Heliospheric imager datasets often use `AZP` or `ZPN` projections. For these datasets, the validation in this note supports the correctness of the `HPC` WCS and sampling path. That conclusion does not extend automatically to `Orthographic` mode, where wide-angle heliospheric images may still not display satisfactorily because their integration into the JHV 3D viewing model remains a separate problem.

A dedicated validator was built in:

- `extra/test/validate_jhv_wcs_against_astropy.py`

Outside the renderer, it reproduces the parts of the JHV WCS/HPC mapping path covered by this note and compares them directly against Astropy wherever the result is fully defined by WCS.

The validator was applied to a variety of real instrument FITS files and metadata, including COR2, HI1, HI2, SDO/AIA, Solar Orbiter/EUI, and PSP/WISPR examples.

The work reported here includes:

- validating `formal-TAN`, `AZP`, and six-term `ZPN` (primary branch only) against Astropy, including round-trip checks.
- validating the `HPC` sampling path against Astropy.

Main conclusions:

- `formal-TAN`, `AZP`, and six-term `ZPN` are validated against Astropy, including the tested inverse mappings.

- The available [AZP](#) validation files are non-slanted ( $\gamma = 0$ ), so slanted [AZP](#) is implemented but not directly tested in the runs reported here.
- JHV [HPC](#) rendering is validated against Astropy for the WCS and sampling path covered by this note.
- direct comparison between the [formal-TAN](#) path in [Orthographic](#) mode and [HPC](#) shows that the two display geometries are not identical even with the same observer viewpoint and [dragRotation](#) = 0.
- the measured discrepancies between [formal-TAN](#) in [Orthographic](#) mode and [HPC](#) are consistent with the theoretical geometric discrepancy derived in Appendix A.
- on the sample TAN files, [simple-TAN](#) remains very close to the [HPC](#) display geometry, which suggests it may be a better choice than [formal-TAN](#) for JHV [Orthographic](#) mode when visual consistency with [HPC](#) is preferred.
- this note is therefore a validation/testing note, not a design note for [Orthographic](#) embedding, playback policy, mouse-position reporting, or synthetic overdrawing.

Bottom line:

- [simple-TAN](#) will be kept in JHV [Orthographic](#) mode for data designated as [TAN](#) in the metadata.
- [HPC](#) mode was added (with the noted caveat about the viewpoint).
- support for [AZP](#) and [ZPN](#) data was added (with the noted caveat about the heliospheric imagers).
- the position numbers reported in the panel at the bottom of the JHV window are display-geometry numbers derived from the mouse pointer position, not coordinates read back from the active image WCS. In [HPC](#), [Latitudinal](#), [Polar](#), and [LogPolar](#), they follow the corresponding JHV display projection. In [Orthographic](#), they are derived purely from the inferred 3D scene point and therefore do not, in general, reflect the image [TAN](#), [AZP](#), or [ZPN](#) WCS.

## 2 The validator

For the validation modes in this note, the validator script provides a Python/CPU model of the relevant JHV reprojection and sampling logic. It does not execute the actual JHV renderer. In particular, major parts of the corresponding code in JHV run in GLSL on the GPU, whereas the validator re-expresses that logic in Python for comparison and testing. It therefore models the reprojection and sampling path, not the full interactive renderer.

### 2.1 JHV behavior modeled by the validator

For each screen pixel, JHV first determines the corresponding point on the display geometry. In `HPC` mode, that geometry is the `HPC` display plane; in `Orthographic` mode, it is the 3D solar scene used by the ortho renderer. From that display point, JHV derives the helioprojective coordinates needed to evaluate the image WCS. For zenithal FITS data in the formal WCS paths, this means using the inverse form of the relevant projection model, such as `TAN`, `AZP`, or `ZPN`, and then applying the linear WCS terms (`CRVAL`, `CRPIX`, `CDELTA`, and `PC/CROTA`) to obtain the source-image coordinates. The screen pixel is then produced by sampling the source image at those coordinates with interpolation. In this sense, the WCS projection determines how image coordinates relate to observer geometry, while the JHV rendering mode determines which display geometry is sampled before that WCS mapping is evaluated.

The sampling pipeline described above, and the point at which `simple-TAN` departs from the formal WCS path, can be summarized as follows:

```
formal WCS:
screen pixel
  -> display geometry point
  -> helioprojective coordinates
  -> WCS world-to-plane step
  -> linear WCS terms
  -> source-image coordinates
  -> interpolated image sample

simple-TAN:
screen pixel
  -> display geometry point
  -> (x,y) coordinates in the image-view plane
  -> linear WCS terms
  -> source-image coordinates
  -> interpolated image sample
```

In `simple-TAN`, the `(x,y)` pair refers to the display point's coordinates in the image-view plane, i.e., the plane perpendicular to the viewing direction in the image observer frame. Those coordinates are used directly as a small-angle approximation to the helioprojective viewing angles, instead of first converting the 3D point to helioprojective coordinates and then applying the `formal-TAN` projection step.

Included in the validator:

- FITS metadata parsing into the same effective quantities JHV uses:
  - CRPIX
  - CRVAL
  - CDELTA
  - PC/CROTA
  - DSUN\_OBS
  - PV2\_0 .. PV2\_5
- the shared WCS projection math for:
  - TAN
  - AZP
  - six-term ZPN (primary branch only)
- the HPC image sampling path:
  - screen HPC coordinate -> helioprojective -> WCS plane -> source pixel
- the raw image-footprint HPC bounds and the centered HPC display bounds used in the validation runs reported here
- direct comparison between the formal-TAN path in Orthographic mode and HPC for the specific no-rotation comparison mode.

Excluded:

- dragRotation
- cameraDiff
- differential diff-image alignment between two arbitrary live layers beyond the specific per-image deltaT reprojection modeled in the shader path
- deltaCROTA, deltaCRVAL1, deltaCRVAL2
- mouse-position reporting or synthetic overdrawing such as annotations, grid overlays, and similar on-screen constructs
- playback/view policy effects such as dynamic refitting of the visible HPC box beyond the centered bounds logic explicitly reproduced by the validator, or any other transient user-driven viewing state
- Java overlay-only behavior such as:
  - viewpoint-space projection of external overlay points in HPC
  - visible-hemisphere clipping of HPC overlay emission
  - transformed annotation drawing/picking behavior
- full OpenGL rasterization state, blending, and UI behavior

The validator therefore establishes:

- whether the implemented reprojection and sampling math matches Astropy
- whether the implemented HPC sampling map matches Astropy
- whether the formal-TAN path in Orthographic mode and HPC sample the same source pixels over the same rendered comparison frame.



## 2.2 Astropy-based validation scope

Astropy can validate the parts of the mapping that are fully defined by WCS.

In this validator, Astropy is the external reference for:

- forward samples
  - JHV-style world/helioprojective -> WCS plane -> pixel
  - checked against `astropy.wcs.WCS.wcs_world2pix(...)`
- full pixel-center validation
  - every source-image pixel center is mapped through the JHV path and checked against Astropy
- inverse TAN
  - plane -> helioprojective, checked by round-trip against Astropy's forward WCS
- inverse AZP
  - plane -> helioprojective, checked by round-trip against Astropy's forward WCS
- inverse ZPN (primary branch only)
  - plane -> helioprojective, checked by the same round-trip strategy on the supported primary monotonic branch
- bounded HPC render comparison
  - the same HPC screen grid is run through the JHV sampling path and through Astropy `world -> pixel`
  - the script compares the resulting source-pixel centers and also writes diff images
- centered HPC bounds reporting
  - the script reports both the raw image-footprint bounds and the centered display bounds used by the JHV HPC screen mapping

Not compared against Astropy:

- direct comparison between the `formal-TAN` path in `Orthographic` mode and HPC
  - this is an internal JHV-to-JHV comparison

## 2.3 How to use the validator

The main script is:

- `extra/test/validate_jhv_wcs_against_astropy.py`

Run it with:

```
python3 extra/test/validate_jhv_wcs_against_astropy.py <fits-file> [mode]
```

The validator uses three different comparison domains:

- full image frame:
  - the actual FITS image pixel grid
- full rendered comparison frame:
  - the full square comparison grid used by the internal *Orthographic*/HPC tests
- bounded HPC screen domain:
  - a finite rendered HPC box chosen for the validation run

### 2.3.1 Astropy-based validation modes

#### 1. Forward WCS random-sample validation

```
python3 extra/test/validate_jhv_wcs_against_astropy.py \  
extra/test/data/20241224_194245_d4c2A.fts
```

This mode reports forward WCS errors on sampled world points. It is not tied to the displayed solar disk or to the full image frame.

It reports:

- *projection\_max\_error\_internal*
- *pixel\_center\_max\_error\_px*

#### 2. Full pixel-center validation

```
python3 extra/test/validate_jhv_wcs_against_astropy.py \  
extra/test/data/20250622_000831_s4h1A.fts \  
--all-pixels
```

This mode checks the full FITS image grid against Astropy and reports the worst pixel-center error.

#### 3. Inverse TAN

```
python3 extra/test/validate_jhv_wcs_against_astropy.py \  
extra/test/data/sample.171.fits \  
--hdu 1 \  
--inverse-tan
```

This mode validates inverse TAN on sampled projection-plane points. It is not an on-disk or full-image-grid test.

It validates:

- TAN plane -> *helioprojective*
- round-trip error

#### 4. Inverse AZP

```
python3 extra/test/validate_jhv_wcs_against_astropy.py \  
extra/test/data/20250622_000831_s4h1A.fts \  
--inverse-azp
```

This mode validates inverse **AZP** on sampled projection-plane points. It is not an on-disk or full-image-grid test.

It validates:

- **AZP plane** -> **helioprojective**
- round-trip error

#### 5. Inverse primary-branch **ZPN**

```
python3 extra/test/validate_jhv_wcs_against_astropy.py \  
extra/test/data/psp_L3_wispr_20231227T150704_V1_2222.fits \  
--inverse-zpn
```

This mode validates inverse **ZPN** on sampled projection-plane points. It is not an on-disk or full-image-grid test.

It validates:

- **ZPN plane** -> **helioprojective**
- round-trip error on the primary monotonic branch

#### 6. HPC render comparison

```
python3 extra/test/validate_jhv_wcs_against_astropy.py \  
extra/test/data/20241224_194245_d4c2A.fts \  
--hpc-render-compare \  
--render-size 2048
```

This mode runs over a bounded **HPC** screen domain and sends the same rendered screen grid through:

- the JHV **HPC** sampling path
- Astropy **world** -> **pixel**

and writes:

- \*\_hpc\_jhv.png
- \*\_hpc\_astropy.png
- \*\_hpc\_diff.png

under:

- **extra/test/out**

It also reports:

- **raw\_bounds\_deg**
- **bounds\_deg**
- **pixel\_center\_max\_error\_px**
- **pixel\_center\_rms\_error\_px**

Interpretation:

- the script reports absolute pixel errors directly
- HI2 can show a slightly larger absolute **px** error because **AZP** magnification near the finite valid edge becomes extremely large

## 2.3.2 Internal JHV comparison modes

### 1. `formal-TAN` vs JHV HPC

```
python3 extra/test/validate_jhv_wcs_against_astropy.py \
  extra/test/data/sample.171.fits \
  --hdu 1 \
  --ortho-vs-hpc-screen-compare \
  --render-size 4096
```

This mode compares, for TAN data:

- the `formal-TAN` path in `Orthographic` mode
- the JHV HPC display sampling path

over the full rendered comparison frame, using a comparison grid at the native image resolution of the tested file.

It writes:

- `*_ortho_screen.png`
- `*_hpc_screen.png`
- `*_ortho_vs_hpc_diff.png`

Interpretation:

- this is not an Astropy comparison
- it measures whether the `formal-TAN` path in `Orthographic` mode and HPC define the same on-screen geometry over that full rendered frame

### 2. `simple-TAN` vs JHV HPC

```
python3 extra/test/validate_jhv_wcs_against_astropy.py \
  extra/test/data/sample.171.fits \
  --hdu 1 \
  --compare-initial-tan-vs-hpc
```

This mode compares, for TAN data:

- `simple-TAN`
- the HPC display sampling path

over the full rendered comparison frame, using a comparison grid at the native image resolution of the tested file.

It writes:

- `*_initial_tan_screen.png`
- `*_hpc_screen_from_initial_tan_compare.png`
- `*_initial_tan_vs_hpc_diff.png`

It also reports:

- `initial_tan_vs_hpc_max_px`
- `initial_tan_vs_hpc_rms_px`

Interpretation:

- this is an internal JHV comparison for **TAN**
- it measures how closely **simple-TAN** in **Orthographic** mode matches the **HPC** display geometry over that full rendered frame

### 3. **simple-TAN** vs **formal-TAN** over the full image frame

```
python3 extra/test/validate_jhv_wcs_against_astropy.py \  
extra/test/data/sample.171.fits \  
--hdu 1 \  
--compare-initial-tan-image-frame
```

This mode compares:

- **simple-TAN**
- **formal-TAN**
- Astropy as the WCS reference

over the full FITS image frame at native image resolution.

It writes:

- \*\_initial\_tan\_image\_frame.png
- \*\_formal\_tan\_image\_frame.png
- \*\_initial\_vs\_formal\_tan\_image\_frame\_diff.png

Interpretation:

- this is an implementation-comparison mode for **TAN** only
- it compares **simple-TAN** and **formal-TAN** against Astropy over the actual FITS image frame and shows where the two JHV paths diverge

## 3 Results

The primary discrepancy units reported in this note are angular sky errors derived from `CDELTA`, expressed in milliarcseconds or arcseconds as appropriate. Pixel errors are kept as secondary units because they are the quantities reported directly by the validator.

### 3.1 FITS projections

These tests compare the implemented JHV projection paths directly against Astropy as the external WCS reference.

1. `formal-TAN` is correct.

- JHV `world` -> `helioprojective` -> `TAN plane` -> `pixel`
- `TAN plane` -> `helioprojective angles`
- matches Astropy to numerical precision
- representative measured results include:
  - forward random-sample check on `20241224_194245_d4c2A.fits`:
    - \* `1.00e-7 mas` max (`6.821210e-12 px`)
    - \* `projection_max_error_internal=1.012523e-13`
  - inverse `TAN` on `sample.171.fits`:
    - \* `2.05e-7 mas` max
    - \* `roundtrip_plane_max_error_internal=8.881784e-16`

2. `formal-TAN` is substantially more accurate than `simple-TAN`.

- native-resolution full-image-frame comparison against Astropy on the same `TAN` samples gives:
  - `sample.171.fits` (`1.009 AU`):
    - \* `simple-TAN`: `2.20 arcsec` max, `1.12 arcsec` RMS (`3.674571 px` max, `1.864828 px` RMS)
    - \* `formal-TAN`: `2.81e-7 mas` max, `6.35e-8 mas` RMS (`4.686171e-10 px` max, `1.059870e-10 px` RMS)
  - `solo_L2_eui-fs174-image_20251002T150055171_V00.fits` (`0.448 AU`):
    - \* `simple-TAN`: `11.42 arcsec` max, `2.46 arcsec` RMS
    - \* `formal-TAN`: `3.41e-7 mas` max, `7.73e-8 mas` RMS
  - `20241224_194245_d4c2A.fits` (`0.967 AU`):
    - \* `simple-TAN`: `2.40 arcsec` max, `0.252 arcsec` RMS

\* `formal-TAN`:  $2.62\text{e-}7$  mas max,  $6.26\text{e-}8$  mas RMS

- for `simple-TAN`, the maximum is set by the on-disk region in all three cases. The RMS values are calculated over the full image frame, which includes both the on-disk sphere region and the off-limb flat-plane region used by the `Orthographic` renderer outside the solar disk. `formal-TAN` remains at numerical precision across the full frame.
- this does not contradict Appendix B. The `simple-TAN` small-angle-approximation error  $\tan(a) - a$  grows with angular distance from the boresight, but this comparison does not measure that error directly everywhere. On the solar disk, `Orthographic` samples from the solar sphere. Outside the solar disk, `Orthographic` uses a flat image-view plane to sample the WCS projection instead. The off-limb part of the comparison is therefore not a direct measurement of the `simple-TAN` small-angle-approximation error alone.

3. `AZP` is correct for the tested non-slanted HI files.

- JHV world -> `helioprojective` -> `AZP plane` -> `pixel`
- `AZP plane` -> `helioprojective angles`
- matches Astropy to numerical precision
- the tested HI files are non-slanted: `PV2_2` is absent, so `gamma` = 0
- JHV also implements slanted `AZP`, but that case is not validated by the sample set reported in this note
- representative measured results include:
  - full pixel-center check on `20250622_000831_s4h1A.fits`:
    - \*  $1.72\text{e-}7$  mas max ( $2.387424\text{e-}12$  px)
  - inverse `AZP` on the same file:
    - \*  $1.02\text{e-}7$  mas max
    - \* `roundtrip_plane_max_error_internal`= $4.263256\text{e-}14$

4. Six-term `ZPN` is correct for the tested PSP/WISPR files.

- JHV world -> `helioprojective` -> `ZPN plane` -> `pixel`
- `ZPN plane` -> `helioprojective angles`
- implemented with `PV2_0..PV2_5`
- matches Astropy to numerical precision on:
  - `extra/test/data/psp_L3_wispr_20231227T150508_V1_1211.fits`
  - `extra/test/data/psp_L3_wispr_20231227T150704_V1_2222.fits`
- the implementation keeps only the primary monotonic branch of the radial polynomial
- a representative measured result on `psp_L3_wispr_20231227T150704_V1_2222.fits`:
  - $1.15\text{e-}5$  mas max
  - `roundtrip_plane_max_error_internal`= $4.840572\text{e-}14$

## 3.2 Astropy validation of JHV HPC rendering

This subsection also compares JHV directly against Astropy, but for the `HPC` sampling path rather than for the FITS projection formulas alone.

For zenithal image data, the deterministic part of the mapping is:

1. image pixel or WCS-plane coordinate
2. inverse WCS ([TAN](#), [AZP](#), [ZPN](#))
3. helioprojective angles
4. observer-frame [HPC](#) ray

That intermediate [HPC](#) ray field is fully determined by FITS/WCS and the observer geometry.

For testing purposes, the validator models this [HPC](#) sampling map over a bounded rendered [HPC](#) screen domain and compares it directly against Astropy.

Measured [HPC](#) validation results:

- COR2 ([TAN](#), 20241224\_194245\_d4c2A.fts) with explicit bounds report:
  - raw image-footprint bounds:
    - \* (-4.479846762614, 4.536935044095, -4.500971194048, 4.495966937161) [deg](#)
  - centered display bounds:
    - \* (-4.536935044095, 4.536935044095, -4.536935044095, 4.536935044095) [deg](#)
  - 1.05e-7 [mas](#) max (7.162271e-12 [px](#))
  - 3.31e-8 [mas](#) RMS (2.254367e-12 [px](#))
- HI1 ([AZP](#), 20250622\_000831\_s4h1A.fts):
  - 3.55e-7 [mas](#) max (4.945377e-12 [px](#))
- HI2 ([AZP](#), 20250622\_000851\_s4h2A.fts):
  - 8.38 [mas](#) max (3.230013e-05 [px](#)) over the finite valid rendered domain
- PSP/WISPR 1211 ([ZPN](#), psp\_L3\_wispr\_20231227T150508\_V1\_1211.fits):
  - 6.65e-6 [mas](#) max (4.365575e-11 [px](#))
- PSP/WISPR 2222 ([ZPN](#), psp\_L3\_wispr\_20231227T150704\_V1\_2222.fits):
  - 1.94e-6 [mas](#) max (9.549694e-12 [px](#))

The centered display bounds are intentionally slightly larger than the raw image-footprint bounds because the JHV [HPC](#) screen mapping recenters the domain symmetrically about disk center and pads the shorter axis to the active display aspect.

For HI2, the finite rendered [AZP](#) domain still produces the largest absolute pixel discrepancy among the tested [HPC](#) render cases because the valid image region extends close to the projection's steep outer edge, where small angular differences translate into larger source-pixel shifts.



### 3.3 Formal-TAN versus JHV HPC

This is not an Astropy correctness test. It is an internal JHV comparison between two different display modes.

Measured comparison result:

- it compares the source pixels sampled by `formal-TAN` in `Orthographic` mode and by `HPC` over the same full rendered comparison frame
- native-resolution full-frame comparison between `formal-TAN` in `Orthographic` mode and `HPC` on the TAN samples gives:
  - `sample.171.fits` (1.009 AU):
    - \* 2.20 `arcsec` max, 1.44 `arcsec` RMS (3.670598 `px` max, 2.397799 `px` RMS)
    - \* theoretical max from Appendix A: 2.19 `arcsec`
  - `solo_L2_eui-fs1174-image_20251002T150055171_V00.fits` (0.448 AU):
    - \* 11.22 `arcsec` max, 7.32 `arcsec` RMS (2.525841 `px` max, 1.649627 `px` RMS)
    - \* theoretical max from Appendix A: 11.05 `arcsec`
  - `20241224_194245_d4c2A.fts` (0.967 AU):
    - \* 2.40 `arcsec` max, 1.56 `arcsec` RMS (1.629523e-01 `px` max, 1.064475e-01 `px` RMS)
    - \* theoretical max from Appendix A: 2.29 `arcsec`

These results support the following conclusions:

- the `formal-TAN` path in `Orthographic` mode and `HPC` are not the same display geometry
- the maximum discrepancy is set by the on-disk part of the frame, so it remains close to the Appendix A prediction
- the visible “bulging” when switching between them is expected from that geometry difference, not from an Astropy/WCS mismatch

### 3.4 Simple-TAN versus JHV HPC

This is again an internal JHV comparison. Its purpose is to show whether `simple-TAN` behaves more like `HPC` or like `formal-TAN` in JHV `Orthographic` mode.

- it compares the source pixels sampled by `simple-TAN` in `Orthographic` mode and by `HPC` over the same full rendered comparison frame
- native-resolution full-frame comparison between `simple-TAN` and `HPC` gives:
  - `sample.171.fits` (1.009 AU):
    - \* 16.8 `mas` max, 4.96 `mas` RMS (2.807831e-02 `px` max, 8.273222e-03 `px` RMS)
    - \* theoretical on-disk max from Appendix B: 6.73 `mas`
  - `solo_L2_eui-fs1174-image_20251002T150055171_V00.fits` (0.448 AU):
    - \* 518 `mas` max, 145 `mas` RMS (1.165703e-01 `px` max, 3.266729e-02 `px` RMS)

- \* theoretical on-disk max from Appendix B: 76.9 mas
- \* large nonzero CRVALi offset the solar disk from boresight and likely contribute to the higher value
- 20241224\_194245\_d4c2A.fts (0.967 AU):
  - \* 21.1 mas max, 5.77 mas RMS (1.434673e-03 px max, 3.928160e-04 px RMS)
  - \* theoretical on-disk max from Appendix B: 7.65 mas
- this confirms that simple-TAN is much closer to the HPC display geometry than formal-TAN, which supports its use in Orthographic mode when visual consistency with HPC is preferred.
- Appendix B gives an idealized on-disk lower bound for the small-angle approximation effect. The measured full-frame maxima are larger, but they should not be read as a stronger version of the on-disk small-angle-approximation error. Outside the solar disk, Orthographic samples all WCS projections from the same flat image-view plane, so the off-limb part of the comparison reflects the difference between that flat Orthographic off-limb geometry and the HPC display geometry.

## 4 Appendix A: Theoretical Orthographic vs HPC discrepancy

This appendix records a purely theoretical geometric discrepancy between:

- a straight-on orthographic projection of the visible solar surface
- a linear HPC display of the same imaged sphere

with both images scaled so that the apparent solar limb radius is the same.

This result is not derived from JHV. It follows directly from the ideal orthographic and HPC projection formulas for a unit solar sphere observed from distance  $D$ .

Assumptions:

- solar radius  $R = 1$
- observer distance  $D$  expressed in solar radii
- HPC is displayed linearly in helioprojective angle
- the comparison is purely geometric and does not depend on JHV-specific rendering choices

Let  $\gamma$  be the heliocentric angle on the solar surface, measured from disk center toward the limb.

The visible limb occurs at:

- $\gamma_{\text{limb}} = \cos^{-1}(1/D)$

The apparent helioprojective limb angle is:

- $\theta_{\text{limb}} = \sin^{-1}(1/D)$

The orthographic and HPC radial coordinates, after equal-limb normalization, are:

- $R_{\text{ortho}}(\gamma) = \sin(\gamma)/\sqrt{1 - D^{-2}}$
- $R_{\text{hpc}}(\gamma) = \tan^{-1}(\sin(\gamma)/(D - \cos(\gamma)))/\sin^{-1}(1/D)$

The normalized discrepancy is therefore:

- $\Delta(\gamma) = R_{\text{hpc}}(\gamma) - R_{\text{ortho}}(\gamma)$

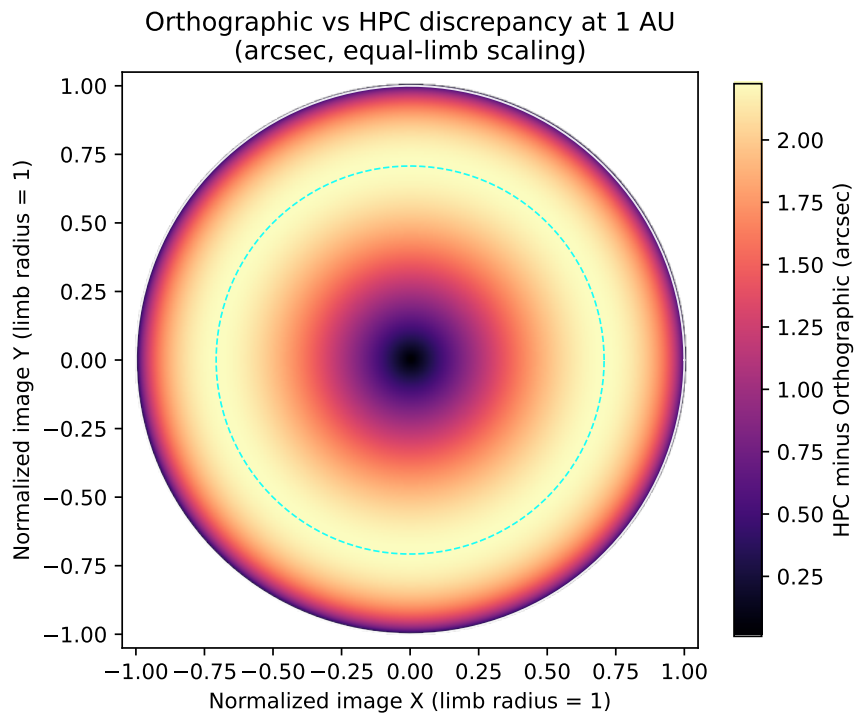
At 1 AU, with  $D \approx 215.03215567$ , the maximum absolute discrepancy is:

- $\max |\Delta| \approx 0.00232016$
- this occurs at  $\gamma \approx 44.84^\circ$
- in angular units, this is about 2.23 arcsec
- equivalently, about 0.232% of the apparent solar limb radius

The discrepancy is positive, so the HPC image bulges outward slightly relative to the orthographic image.

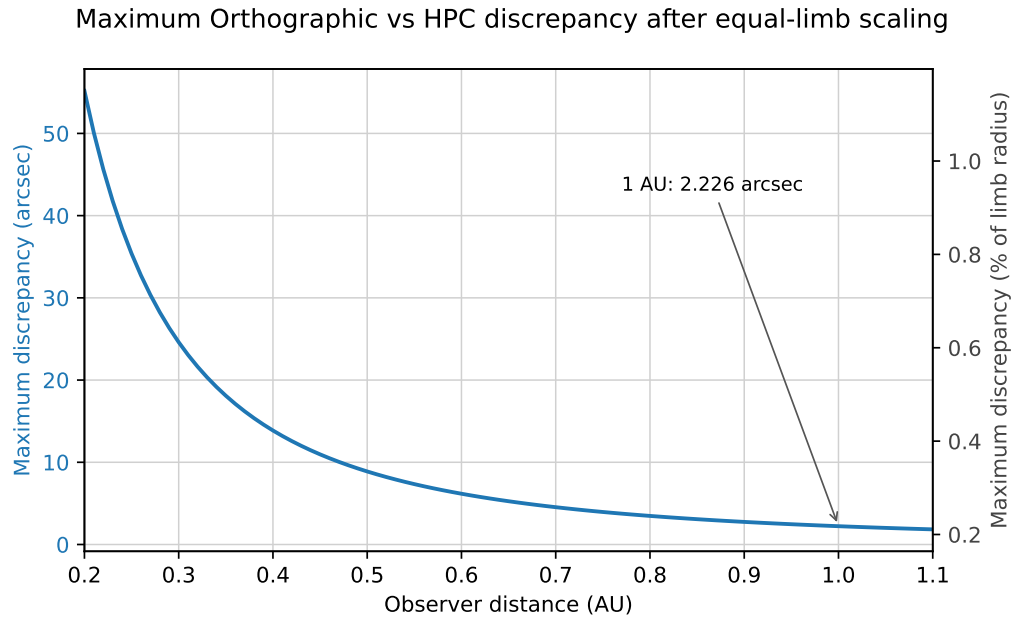
The discrepancy is zero at disk center and at the limb, and peaks on a ring at about  $44.8^\circ$  heliocentric angle, corresponding to about 70.7% of the apparent disk radius.

The following figure shows the same discrepancy at 1 AU as a radial gradient over the normalized solar disk. The dashed ring marks the location of the maximum discrepancy.



**Figure 4.1:** Orthographic vs HPC discrepancy across the projected solar disk at 1 AU

The following figure shows the maximum discrepancy as a function of observer distance between 0.2 AU and 1.1 AU.



**Figure 4.2:** Maximum Orthographic vs HPC discrepancy versus observer distance

## 5 Appendix B: Idealized TAN small-angle discrepancy relative to HPC

This appendix records a different theoretical discrepancy from Appendix A. It isolates one effect only: the error introduced when TAN is approximated by the small-angle relation  $\tan(a) \approx a$ . This is a purely angular error. It is not derived from JHV. The centered solar-disk case considered below is only one example of it.

Assumptions:

- the relevant angular distance is measured from the instrument boresight
- HPC is displayed linearly in helioprojective angle
- the exact TAN plane coordinate is replaced by its small-angle approximation

Let  $a$  be the helioprojective angular distance from the instrument boresight. In linear HPC, the radial coordinate is proportional to  $a$ . In TAN, the radial coordinate on the projection plane is proportional to  $\tan(a)$ . The corresponding continuous discrepancy is therefore:

- $\Delta(a) = \tan(a) - a$

Unlike Appendix A, this discrepancy is monotonic in  $a$ : the function  $\tan(a) - a$  increases as  $a$  increases. The discrepancy therefore grows with angular distance from the instrument boresight.

To relate this purely angular error to a solar image, consider the special case of a centered solar disk observed from distance  $D$  expressed in solar radii. Then the solar-limb angle is:

- $a_{\text{limb}} = \sin^{-1}(1/D)$

so the maximum discrepancy is:

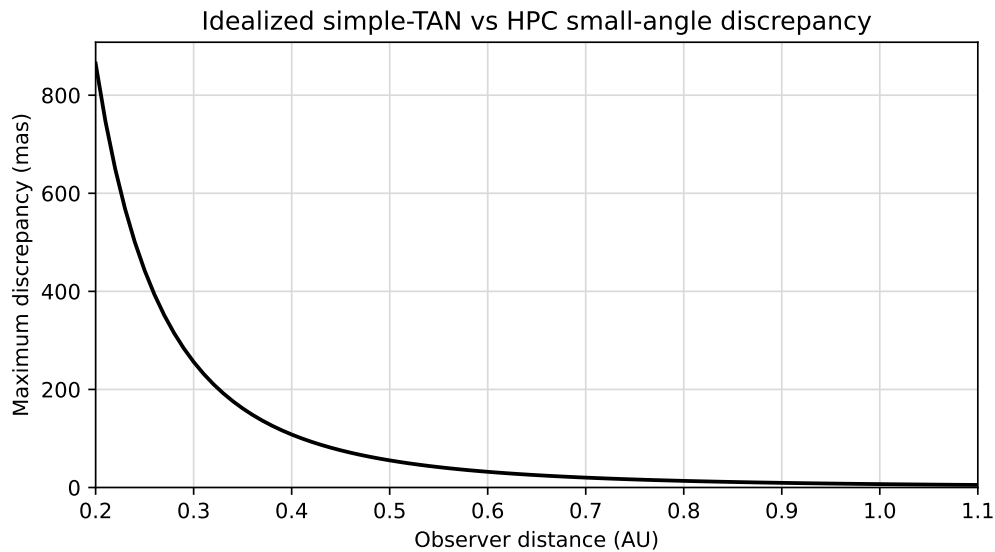
- $\Delta_{\text{max}}(D) = \tan(\sin^{-1}(1/D)) - \sin^{-1}(1/D)$

At 1 AU, with  $D \approx 215.03215567$ , this gives:

- $\Delta_{\text{max}} \approx 3.35 \times 10^{-8}$  rad
- about 6.92 mas

At 0.2 AU, the same idealized discrepancy is about 865 mas.

The following figure shows this centered-disk example of the idealized maximum discrepancy as a function of observer distance between 0.2 AU and 1.1 AU.



**Figure 5.1:** Maximum idealized TAN small-angle discrepancy relative to HPC versus observer distance

In the centered-disk example treated here, disk center coincides with the boresight, so the discrepancy increases continuously from solar disk center to the limb, where the on-disk maximum occurs. If the solar disk is offset from the boresight, the on-disk maximum is larger because part of the disk lies at larger boresight angles.

For coronagraph images, where the field of view extends to much larger angles than the solar disk, the same angular small-angle-approximation error can reach hundreds of arcseconds near the image-frame edge.